

Class 2: What is Python?

Data-driven Systems Engineering

Agenda

- Python's role in the data and ML ecosystem.
- Interpreted, high-level, object-oriented: what that actually means.
- Why Python became the default language for data-driven engineering.
- A first tour of the toolchain this course reuses: Jupyter, pandas, scikit-learn, Flask, Streamlit.
- Where each tool already lives inside this repository.

Learning goals

- Describe Python as an interpreted, high-level, general-purpose, object-oriented language.
- Contrast interpreted execution with compiled execution and explain the trade-off.
- Explain why one language can cover analysis, services, and interfaces in this course.
- Name the five core tools in the course stack and the job each one performs.

Course Map and Running Cases



Today in the course

Focus: Python as the shared working language for every remaining session.

Bridge: Before we touch data or models, we need one reliable programming substrate for exploration, services, and interfaces alike.

Fraud case

The fraud-detection code students read starting a few classes from now — ‘streamlit/modeling.py’, the Streamlit dashboard, the credit-card scoring page — is ordinary Python: imports, functions, and objects, nothing fraud-specific about the language itself.

Pasteurization case

The pasteurization simulator and its streaming service are also just Python: the same interpreter and object model, wired to synthetic sensors instead of a transaction feed.

The new age of programming

- Software has changed dramatically in 30 years: from early personal computers to phones, tablets, and always-on services.
- The internet moved software from desktop-only to web and mobile delivery.
- Thousands of programming languages exist today; the market rarely settles on just one.
- Data-driven engineering added a further requirement: a language comfortable with both messy data and production services.

What makes Python, Python

Property	What it means
Interpreted	Code runs from source through an interpreter; no separate compile-to-machine-code step.
High-level	Memory management and low-level detail are hidden from the programmer.
General-purpose	The same language covers scripts, services, analysis, and applications.
Object-oriented	Data and behavior can be bundled together in classes and objects.
Open	Free, community-driven, maintained by the Python Software Foundation.

- Created by Guido van Rossum, first released in 1991.

Interpreted versus compiled

Compiled

- Source is translated to machine instructions ahead of time.
- Produces a standalone executable.
- Generally faster at run time.
- Common for permanent, performance-critical applications.

Interpreted

- Source is kept in the format you wrote it in.
- Parsed and executed every time it runs.
- Slower per instruction, but far faster to iterate on.
- A natural fit for ad hoc analysis, calculation, and simulation.

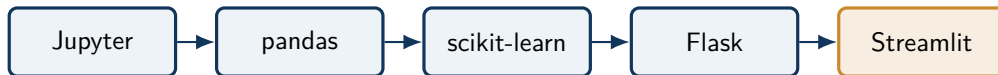
Course relevance

EDA, prototyping, and gluing services together all reward fast iteration over raw execution speed — exactly what an interpreted language optimizes for.

Why data-driven systems engineering needs Python

- Data-driven engineering means system behavior depends partly on data and learned patterns, gathered from sensors, logs, and user interactions.
- Its core activities — data collection and integration, modeling and analysis, optimization, real-time monitoring, decision support, automation — all need a language with libraries for every step.
- Python is the rare language with mature libraries for each of those activities, plus web frameworks to expose the results.
- Packages bring flexibility and a large community, at the cost of installation and version management overhead — a trade-off worth knowing early.

The toolchain this course reuses



- Explore and teach in Jupyter, manipulate tables in pandas, model with scikit-learn, serve with Flask, and put an interface in front of it with Streamlit.
- Every one of these already appears in this repository, not only in slides.

What each tool is actually for

Tool	Job
Jupyter	Interactive notebooks for exploration, teaching, and narrated analysis.
pandas	Loading, inspecting, and reshaping tabular data.
scikit-learn	Preprocessing, training, evaluating, and persisting classical ML models.
Flask	A lightweight web framework to expose predictions as an API.
Streamlit	A rapid way to turn a Python script into an interactive dashboard.

Python you will recognize by the end of this class

```
import streamlit as st

st.set_page_config(
    page_title="440MI - Machine Learning Dashboard",
    page_icon=":robot_face:",
    layout="wide",
    initial_sidebar_state="expanded"
)

st.title("Machine Learning Streamlit Hub")
```

- This is the opening of 'streamlit/Main.py', the entry point of the course's dashboard.
- Nothing here is exotic: an import, a function call with keyword arguments, and a method call — core Python syntax we build up starting next class.

Where the toolchain already lives in this repository

Repo activity

- 'requirements.txt' pins pandas, scikit-learn, matplotlib, seaborn, Flask, and lists streamlit and numpy among the project dependencies.
- 'notebooks/' holds the Jupyter material we will use for EDA and model training.
- 'streamlit/Main.py' and 'streamlit/pages/' are the dashboards the fraud case will run through.
- 'streamlit/modeling.py' trains and pickles the fraud model students will later serve and inspect.

Setting up for this course

1. Install Python (3.x) and a package manager, or use a distribution that bundles both.
2. Create an isolated environment for the course so package versions do not collide with other projects.
3. Install the packages listed in 'requirements.txt'.
4. Confirm Jupyter starts and a notebook in 'notebooks/' runs top to bottom.

Questions you should be able to answer after Class 2

- Why is Python described as interpreted, high-level, and object-oriented, and what does each word actually change?
- What is the practical trade-off between compiled and interpreted execution?
- Name one job each of Jupyter, pandas, scikit-learn, Flask, and Streamlit does in this course.
- Where in this repository does each of those five tools already appear?

Question

If Python is slower per instruction than a compiled language, why does it dominate data-driven engineering rather than losing out on performance grounds?

- Pushes students to separate developer productivity from raw execution speed.
- Sets up the idea that critical inner loops (NumPy, scikit-learn) are themselves compiled, while Python orchestrates them.

Papers and materials

Jupyter install guide; pandas documentation; scikit-learn documentation; Flask documentation; Get started with Streamlit.

From This Class to the Next

What stays with us

- Python is interpreted, high-level, general-purpose, and object-oriented — a deliberate combination, not an accident.
- The trade-off of interpreted execution is worth it for the iteration speed that data-driven work needs.
- Jupyter, pandas, scikit-learn, Flask, and Streamlit already exist in this repository and will reappear all semester.

Project use

Before writing a line of your own project code, confirm your environment can run the notebooks and dashboards already in this repo.

Next connection

Class 3 starts writing real Python: variables, types, and the small syntax rules every later script depends on.

Course thread: Every tool tour eventually turns into code we read line by line — that starts next class.